

How COMMAND Analytics works

A technical walkthrough, with the code that does it

docs/use-case-2/21-under-the-hood-brief.md · rendered for review

This note explains how COMMAND Analytics works under the hood, for a reader who knows ServiceNow but does not write ServiceNow code. It ships alongside the source, so every claim names the file and line where the code can be found — `CmdData.js:369` means line 369 of that file. Script files live in `product/script-includes/`, browser files in `product/ui-scripts/`, pages in `product/ui-pages/`, the installer in `product/deploy/`. A glossary of platform terms sits at the end.

Where the code lives

The whole product is twelve files. There is no server outside ServiceNow and no database outside ServiceNow; everything below is installed on the instance as ordinary configuration records.

FILE	LINES	WHAT IT IS RESPONSIBLE FOR
<code>CmdData.js</code>	2,918	every read of a row, and the permission proof behind every number
<code>CmdPayload.js</code>	1,471	assembles one dashboard: which panels exist and in what order
<code>CmdAnalysis.js</code>	1,409	the richer panels — trends by category, backlog, waterfall, box plot
<code>CmdCeo.js</code>	1,246	redraws a Performance Analytics dashboard as a COMMAND page
<code>CmdReport.js</code>	628	redraws a saved ServiceNow report without changing what it counts
<code>CmdForm.js</code>	606	chooses the visual form from the measured shape of the data
<code>CmdCatalog.js</code>	586	the entry page: which subjects a given viewer may open
<code>CmdMeta.js</code>	469	reads the schema, so no table or field is ever named in code
<code>CmdDrill.js</code>	405	decides which drilldown levels are real, and which are not
<code>cmd_render.js</code>	4,200	draws every chart in the browser, from the embedded data
<code>cmd.css</code>	690	the shared stylesheet for both pages
<code>cmd_dashboard.html</code> , <code>cmd_catalog.html</code>	297	the two pages themselves

How one page is built

A viewer opens `cmd_dashboard.do?table=incident`. The platform runs `CmdPayload.dashboard` (`CmdPayload.js:163`), which measures the table, decides what is worth drawing, counts it, and returns a single structure holding every number the page needs. That structure is embedded directly into the page's HTML (`cmd_dashboard.html:27` computes it, `:169` embeds it) and the browser draws it.

It is one round trip, and that is forced rather than preferred. On this instance a request from a logged-in browser session to a Scripted REST endpoint never returns, and the Table API behaves the same way — recorded at the head of `CmdPayload.js` and in `poc/servicenow/ui-page/probe-results.md`. Nothing is fetched after the page loads; the page arrives finished.

The data travels as base64 text, because a payload carries field labels and record values that could contain the sequences the platform's template language evaluates. Encoding removes the possibility (`cmd_dashboard.html:20`, encoded at `:158`).

Nothing is exported. No data leaves the instance at any point, and the page makes no request to any outside host — no fonts, no chart library, no analytics. The fonts are embedded in `cmd_fonts.js` and the chart code is written in-house.

1. Every number is counted as the viewer, not as an administrator

This is the part of the product that matters most, and it exists because of a real gap in the platform.

ServiceNow's fast counter, `GlideAggregate`, **does not apply row-level permissions**. It will happily count rows the viewer is not allowed to open. There is no secure variant of it, and the obvious alternative — `GlideQuery.withAcls()` — raises an error rather than aggregating. A dashboard built the straightforward way therefore shows every viewer the administrator's numbers.

So every page runs both counts and compares them. `CmdData.aclVerdict` (`CmdData.js:369`) is where that happens:

```
var fast = this.fastCount(table, query); // CmdData.js:402
var proof = this.secureCountBoxed(table, query, CmdData.PROOF_MS); // :469
```

The first is the indexed aggregate — fast, and blind to permissions (`fastCount`, `CmdData.js:691`). The second walks the rows through `GlideRecordSecure`, which is the platform's own permission check, inside a time budget (`secureCountBoxed`, `CmdData.js:563`). **The difference between the two numbers is exactly the set of rows the viewer cannot open.**

The verdict is built at `CmdData.js:478` and shown on every page as a badge in the top right corner:

BADGE	MEANING
ACL VERIFIED	both counts agree — nothing is being hidden from this viewer
FILTERED	the viewer sees fewer rows than exist, and the page says how many
BOUNDED	the check ran out of time, so the number shown is a floor, not a total
NO ACCESS	the viewer may not read any of these records

Measured on the development instance on **10 September 2026**, acting as a real user holding no roles, with no change made to the instance's security setup:

TABLE	WHAT NATIVE REPORTING COUNTS	WHAT THIS USER CAN ACTUALLY OPEN
incident	4,284	818
task	8,503	818
problem	544	0
change_request	1,505	0
kb_knowledge	757	669
sys_user	667	667 (the control)

Read the first row again: **native reporting shows that user 4,284 incidents when they can open 818** — an overstatement of 3,466 records. On `problem` a COMMAND page shows a red **NO ACCESS** badge over *"This subject holds 544 records and your permissions do not admit any of them, so there is nothing to show."*

A completed proof is remembered for three minutes per viewer and per query (`CmdData.js:282`, `_rememberTrust` at `:509`) so that a second page for the same person does not pay for it again. Only a proof that finished is ever cached; a bounded one is not, because a floor must not be mistaken later for a total.

2. Finding what to show, with no per-table configuration

The product works over ITSM, HR, assets, knowledge, GRC and anything else without a line of setup per table, because `CmdMeta.js` reads `sys_dictionary` at request time and derives everything: `describe` (`CmdMeta.js:83`) identifies a table, `dimensions` (`:284`) works out which columns can be grouped, `dates` (`:352`) which can carry a trend, `measures` (`:365`) which can be summed or averaged.

Nothing in the codebase names a table or a field. A new table appears in the catalog with nobody configuring it, and a customer schema that differs from the demo instance does not break anything.

3. The chart is chosen by measuring the data, not from a stored list

Most tools save a chart type with the report — someone picked *pie* once and it is a pie forever, whatever the data later becomes. COMMAND measures the column first and then picks a form that shape can carry honestly.

The measurement happens in `CmdData._shape` (`CmdData.js:1229`): how many distinct values the column holds, how dominant the largest one is, and how populated the column is overall.

```
var distinct = rows.length; // CmdData.js:1232
var topShare = (total > 0 && distinct > 0) ? rows[0].count / total : 0;
fill: total > 0 ? (total - emptyCount) / total : 1, // :1288
```

`CmdForm.selectForm` (`CmdForm.js:115`) then walks a ladder of questions. A few rungs, to show the character of it:

- one distinct value is not a distribution, so it becomes a single number (:134)
- two time periods across several categories is a before-and-after, not a trend, so it becomes a slope chart rather than a line that invites reading a trajectory from two points (:147)
- a level that exists at each moment, such as open backlog, gets a filled area because the area to zero means something; a count of events per period gets a bare line, because there the area would overstate it (:158)
- four series that sum to a whole become a share-over-time chart; more than can be read as overlapping lines become a grid of small charts (:162 – :171)

After a form is chosen, `applyGuards` (`CmdForm.js:258`) can still demote it, and records why. This is the part worth showing a client, because the reason travels on to the page:

```
if (c.zeroVariance && form !== 'stat_tile') { // CmdForm.js:344
  form = 'stat_tile';
  demoted = true;
  reason = 'every category holds the same value, so the distribution carries no information';
```

`alternatives` (`CmdForm.js:485`) computes the other forms the same data could honestly take, which is what fills the small form-picker on each panel — so the viewer can change the chart, but only to something the data supports.

The practical result: one subject draws different charts as its data changes. Six categories become a share-over-time chart, eleven become a grid of small charts, and a column that has collapsed to one value becomes a single number with a sentence explaining itself.

4. Drilldown is offered only where the data supports it

ServiceNow's dictionary declares which fields are parent and child. A declaration is not a fact about the rows: on one tenant `incident.subcategory` was declared under `category` and filled in on **42 of 13,986** records — a drill that leads nowhere 99.7% of the time. Following declarations blindly produces a dashboard full of clicks that land on "(none)".

So every candidate level is measured against the viewer's own permitted rows first. The thresholds sit in one place, `CmdDrill.GATES` (`CmdDrill.js:40`): at least 2 distinct values, no more than 50, populated on at least 60% of the slice. `CmdDrill.gate` (`CmdDrill.js:101`) applies them cheapest-first, and a refused level carries its reason instead of being a dead click.

One rule inside that method is worth stating on its own, because getting it wrong produced the most serious defect found in this project:

```
// CmdDrill.js:159 – a bounded scan proves presence and never absence
if (!prof.measured) {
  res.reason = 'not measured: the permission-checked scan ran out of ' +
    'time on this subject before it read any rows, so ' +
    'whether this is a level is unknown rather than no';
```

Before that branch existed, a permission check that ran out of time before reading a single row returned "zero distinct values", and the gate reported that as *"every record here has the same active"* — about a column holding 7,380 true and 1,123 false. It had turned *"we did not look"* into *"there is nothing there"*, in the same calm voice it uses for real measurements.

A measurement of nothing must never become a claim about everything. The asymmetry is now explicit in the code: a partial scan can prove a value is *present*, never that one is *absent*. `CmdData._shape` marks a scan that read no rows as unmeasured (`CmdData.js:1273`), the gate refuses to draw a negative conclusion from it, and `product/tests/test_drill_gates.js` holds 25 assertions that keep it that way.

The last step of any drill path is the platform's own list view (`CmdDrill.listUrl` , `CmdDrill.js:383`). That is deliberate: the list enforces row-level permissions itself, so COMMAND neither builds a record grid nor has to get its security right.

5. The catalog decides what a viewer may even see

The entry page is a catalog of subjects, not a single dashboard (`CmdCatalog.build` , `CmdCatalog.js:190`). Membership is a permission check rather than a guess: a subject appears only if the viewer can actually read enough of it, proved by a bounded secure scan.

Asking the table whether it is readable *in principle* is not the same question, and the difference is documented at `CmdCatalog.js:240` . ServiceNow's `canRead()` evaluates the read rules with no record in context, so a rule whose condition mentions the record fails for a viewer who can nonetheless read plenty of rows. Trusting it removed the `incident` and `kb_knowledge` cards from a user holding 818 and 669 readable records.

6. Redrawing the CEO Dashboard

The client's CEO Dashboard is not a report. It is a Control Tower experience whose pages hold no query at all — each card is a reference to a Performance Analytics *indicator*. `CmdCeo.js` resolves those into something computable: `resolve ( CmdCeo.js:275 )` follows an indicator through its cube to a real table and filter, `parseFormula ( :384 )` handles the ones that are arithmetic over other indicators, and `value ( :482 )` computes the result — through `CmdData`, so a redrawn card inherits the same permission proof as every other number in the product.

That is the point of redrawing rather than screenshotting: Performance Analytics scores bypass row-level permissions and these do not, so for any non-administrator the two differ — and the COMMAND number is the one matching what they can open.

7. Staying inside a time budget, and saying so when it cannot

Permission-checking row by row is correct and slow. Rather than let a page run for as long as the data demands, the product gives the whole request one budget — `CmdData.SCAN_ALLOWANCE_MS`, 6.5 seconds (`CmdData.js:252`) — and spends it.

When it runs out the page neither hangs nor quietly shows a wrong number: it draws fewer panels, marks its counts as a floor, and says what it skipped and why. The comment at `CmdData.js:216` records the measurements behind the figure, including a case where the identical page produced eleven panels on one engine and nine on another because the budget was too tight to absorb the difference.

8. How it reaches the instance, and how the team knows it landed

A ServiceNow write answers **HTTP 200 whether or not it stored what was sent**. Two rounds of this project were lost to reporting a fix as deployed while the instance still served the old code, so the installer never consults the status code.

Every write is read back and compared byte for byte (`sncClient.py:273`). Every page is parsed first, because a page whose template does not compile is served as zero bytes with no error anywhere (`deploy.py:90`). Every script is checked for language features the platform's engine lacks (`deploy.py:120`), and every asset fetched from the exact URL the page will request (`deploy.py:280`). And because a compile error leaves a record byte-perfect while the class fails to load, the deploy ends by asking the instance to construct each one (`deploy.py:155`):

```
js.append(                                     # deploy.py:169
    "try { new %s(); out['%s'] = 'ok'; } "
    "catch (e) { out['%s'] = String(e).substring(0, 160); }" % (n, n, n)
)
```

Glossary

- **ACL** — access control list; the platform's row-level permission rules
- **GlideAggregate** — ServiceNow's fast counter. Ignores row-level permissions
- **GlideRecordSecure** — reads rows *with* permissions applied. Correct, and slower
- **Script Include** — a server-side code record. The nine `Cmd*` files are these
- **UI Page** — a server-rendered page record. The two COMMAND surfaces are these
- **UI Script** — a browser-side code record, served to the page as a file
- **Encoded query** — ServiceNow's text filter format, e.g. `category=software`
- **Indicator** — a Performance Analytics metric definition
- **Dimension** — a column values can be grouped by, such as Category or Priority
- **Fill rate** — the share of records where a column actually has a value
- **Cardinality** — how many different values a column holds

What it does not do — stated plainly, so nothing surprises anyone later

- **It is not a Power BI replacement.** Power-BI-grade visual and interaction quality on ServiceNow-native data — but no joining across outside systems, and no free-form self-service modelling.
- **AI is optional and off by default.** Nothing on these pages is generated by a language model today. Switched on, it would see column statistics and the totals already on screen — never a record.
- **On large tables it is honest rather than fast.** Where row-by-row permission checking would take too long, the page stops and shows **BOUNDED** rather than a quiet count the viewer is not entitled to.